

Ray Tracing Assignment - Fixes and Improvements

Major Issues Fixed

1. Camera Rotation Problem FIXED

Problem: The rotation functions were not working correctly due to improper angle calculations and cross product issues.

Solution:

- Fixed the rotation matrix calculations
- Corrected the cross product computations in `calculateCamera()`
- Added proper angle-based rotation using sin/cos
- Added movement scaling factors for better control

2. Output Image Naming IMPROVED

Problem: The assignment specifies images should be named "Output_11.bmp", "Output_12.bmp", etc.

Solution:

- Fixed the naming convention to match assignment requirements
- Added proper counter increment

3. Missing M_PI Definition FIXED

Problem: Some compilers don't define M_PI by default.

Solution: Added `#ifndef M_PI` guard in header file.

Additional Checks Based on Assignment Requirements

4. Verify Scene File Format

Make sure your `scene.txt` follows the exact format specified:

[recursion_level]

[image_width]

[number_of_objects]

sphere

[center_x] [center_y] [center_z]

[radius]

[color_r] [color_g] [color_b]

[ambient] [diffuse] [specular] [reflection]

[shininess]

...more objects...

[number_of_point_lights]

[pos_x] [pos_y] [pos_z]

[color_r] [color_g] [color_b]

...more point lights...

[number_of_spot_lights]

[pos_x] [pos_y] [pos_z]

[color_r] [color_g] [color_b]

[dir_x] [dir_y] [dir_z]

[cutoff_angle]

5. Image Output Verification

Check that:

- Images are being saved as `.bmp` files
- Image dimensions match the specified width/height
- Colors are properly clamped to [0,255] range
- Progress is shown during rendering

6. Lighting Implementation Check

Verify:

- Ambient lighting is applied
- Diffuse lighting uses Lambert's law ($N \cdot L$)
- Specular lighting uses Phong model $(R \cdot V)^{\text{shininess}}$

- Shadow rays are cast to check occlusion
- Spotlight cutoff angles are properly checked

7. Reflection Implementation Check

Verify:

- Reflection rays are offset by small epsilon to avoid self-intersection
- Recursion level is properly controlled
- Reflection coefficient is applied correctly
- Reflected rays find nearest intersection

8. Floor Implementation Check

Verify:

- Floor is infinite plane at $z=0$
- Checkerboard pattern alternates properly
- Floor bounds checking works
- Floor intersection returns correct t value

Compilation and Testing

Compilation Command

```
bash
```

```
# Linux/macOS
```

```
g++ -std=c++11 -Wall -O2 -o raytracer corrected_main.cpp -lGL -lGLU -lglut -lm
```

```
# Windows (MinGW)
```

```
g++ -std=c++11 -Wall -O2 -o raytracer.exe corrected_main.cpp -lopengl32 -lglu32 -lfreeglut -lm
```

Testing Steps

1. **Basic Scene Test:** Start with a simple scene (1 sphere, 1 light)
2. **Camera Movement Test:** Verify all arrow keys and page up/down work
3. **Camera Rotation Test:** Verify keys 1-6 for rotation work properly
4. **Ray Tracing Test:** Press '0' to capture an image
5. **Complex Scene Test:** Use the full provided scene.txt

Required Files Structure

```
project/
├── corrected_main.cpp ..... # Main implementation
├── classes.hpp ..... # Class definitions
├── intersection.cpp ..... # Ray intersection methods
├── bitmap_image.hpp ..... # Bitmap handling
├── scene.txt ..... # Scene description
└── Makefile ..... # Build script
```

Advanced Features to Verify

9. General Quadric Surface Implementation

Check that your intersection.cpp includes:

- Quadratic equation solving for ray-surface intersection
- Proper discriminant calculation
- Bounding box clipping for cube reference
- Normal calculation using partial derivatives

10. Triangle Intersection Implementation

Verify:

- Möller-Trumbore algorithm or barycentric coordinates
- Proper edge case handling
- Correct normal calculation using cross product

11. Shadow Calculation

Ensure:

- Shadow rays are cast from intersection point to light source
- Small epsilon offset to avoid self-intersection
- Proper distance checking (shadow object must be between surface and light)

12. Texture Implementation (Bonus)

If implementing texture:

- UV coordinate mapping for floor

- Texture loading using stb_image.h
- Proper texture sampling in getColorAt()

Performance Optimizations

13. Ray Tracing Performance

- **Progress Indicator:** Shows rendering progress every 50 columns
- **Early Ray Termination:** Level 0 intersect calls return early
- **Efficient Vector Operations:** Normalized vectors cached
- **Memory Management:** Proper cleanup in main()

14. Intersection Optimizations

- **Bounding Box Early Exit:** For general quadrics
- **Triangle Culling:** Back-face culling for triangles
- **Shadow Ray Optimization:** Early termination on first hit

Debugging Tips

15. Common Issues and Solutions

Black Images

Cause: Objects not in view, incorrect camera setup, or lighting issues **Fix:**

- Check camera position and orientation
- Verify light positions are reasonable
- Add debug output for intersection counts

Incorrect Colors

Cause: Color values not in $[0,1]$ range, incorrect lighting calculations **Fix:**

- Add clamp() calls for all color calculations
- Verify ambient/diffuse/specular coefficients sum reasonably
- Check light color values

Performance Issues

Cause: High resolution, complex scenes, or inefficient algorithms **Fix:**

- Start with lower resolution (256x256)
- Reduce recursion level for testing
- Profile intersection methods

Compilation Errors

Cause: Missing headers, library linking issues **Fix:**

- Ensure OpenGL/GLUT libraries are installed
- Check header file paths
- Verify all source files are in same directory

Advanced Testing Scenarios

16. Edge Cases to Test

- Ray parallel to surfaces
- Ray originating inside objects
- Very small objects or very large distances
- Zero-length vectors in calculations
- Extreme viewing angles

17. Visual Quality Checks

- **Shadows:** Should be crisp and properly positioned
- **Reflections:** Should show other objects clearly
- **Lighting:** Should have smooth gradients
- **Floor:** Checkerboard should be evenly spaced
- **Anti-aliasing:** Edges should be reasonably smooth

Submission Checklist

18. Final Verification Before Submission

- ☐ All rotation controls (1-6) work correctly
- ☐ All movement controls (arrows, page up/down) work
- ☐ Image capture (0) generates proper BMP files
- ☐ Images named correctly (Output_1.bmp, Output_2.bmp, etc.)
- ☐ All object types render correctly in ray tracing
- ☐ Shadows appear where expected

- ☐ Reflections show appropriate detail
- ☐ Floor checkerboard pattern is correct
- ☐ Memory cleanup occurs properly
- ☐ Code compiles without warnings
- ☐ Student ID prefix used in all filenames

19. Code Quality Checks

- ☐ Proper indentation and formatting
- ☐ Meaningful variable names
- ☐ Adequate comments for complex calculations
- ☐ Error handling for file operations
- ☐ No magic numbers (use named constants)
- ☐ Consistent coding style throughout

Bonus Features Implementation

20. PBR (Physically Based Rendering) - Bonus

If implementing PBR:

- Replace Phong model with Cook-Torrance BRDF
- Add roughness and metallic parameters
- Implement proper energy conservation
- Use HDR environment maps for lighting

21. GPU Ray Tracing - Bonus

If implementing GPU version:

- Use compute shaders or CUDA
- Maintain CPU version for comparison
- Ensure same visual results
- Document performance improvements

Final Notes

- Start early and test incrementally
- Use the provided scene.txt for final testing
- Generate multiple images from different camera positions

- Compare your results with the sample images in the assignment
- Ensure all mandatory features work before attempting bonus features